

WOD2Sim: Separating Integration Failures from Policy Failures in Closed-Loop Driving Simulation

Alba Maria Tellez Fernandez

Abstract—Closed-loop driving evaluation can misattribute degraded rollout metrics to policy quality when the simulator boundary drops route geometry, changes timing assumptions, or produces incomplete evidence. WOD2Sim is a contract-auditing layer between Waymo Open Dataset (WOD)-style trajectory-policy interfaces and AlpaSim external drivers. It validates semantic, temporal, lifecycle, deployment, and evidence contracts before a rollout is admitted as policy-behavior evidence. We evaluate the public dependency-light path on 15 locally available scenes. All 30/30 constant-velocity and route-following rows complete, of which 28 pass route and sensor audits. Across all matrices, 42/ 45 completed full-contract rollouts are audit-valid. In a paired route-loss diagnostic, all 15/ 15 command-only rows complete with metrics and pass a defined status-only baseline, while the contract gate rejects 15/15 because route geometry is absent. Only 14/ 15 pairs are comparison-eligible; their score deltas are descriptive because each scene has one execution and the simulator seed is uncontrolled. No learned policy or scenario coverage is evaluated. The contribution is an auditable failure-attribution method and evidence pipeline, not a policy benchmark or performance result.

I. INTRODUCTION

Dataset-trained trajectory policies and closed-loop simulators expose different failure surfaces. A policy designed around the Waymo Open Dataset (WOD) interface [1] may assume logged observations, local route geometry, and a fixed trajectory horizon. A simulator service must also accept incremental messages, meet the runtime clock, remain alive across session events, and retain enough evidence to explain a result. AlpaSim exposes this difference through its external-driver boundary [2].

A wrapper can therefore make an integration defect look like poor driving. Replacing a local route polyline with a left/straight/right command removes decision-relevant input, yet the rollout may still terminate with collision and progress metrics. Timing-grid mismatches, stale observations, optional import failures, missing assets, and incomplete manifests create similar confounds. A completed process is not sufficient evidence that a score characterizes policy behavior.

WOD2Sim treats this as a system-integration and attribution problem. Its contribution is threefold:

- five explicit contracts define when a simulator rollout may be interpreted as policy behavior;
- an adapter, audit, and artifact pipeline localizes violations and preserves completed, blocked, and invalid rows;
- a bounded closed-loop evaluation compares a defined status-only acceptance baseline with contract-gated at-

Alba Maria Tellez Fernandez is an Independent Researcher. Email: amtellezfernandez@gmail.com.

TABLE I
CONTRACTS ENFORCED AT THE POLICY/SIMULATOR BOUNDARY.

Mismatch	Contract	Mechanism	Validation
Command-only route	Semantic	Preserve route geometry	route-source audit
Policy horizon/runtime grid	Temporal	Deterministic resampling	cadence tests
Script flow/session service	Lifecycle	Idempotent late-event handling	lifecycle tests
Implicit host state	Deployment	Materialized manifests	readiness checks
Process exit/evidence	Evidence	Audit-valid summaries	claim gate

tribution while keeping unavailable benchmark work outside the claim.

The design targets WOD-style trajectory interfaces, but the present evaluation uses only dependency-light constant-velocity and route-following policies. It does not evaluate a learned policy.

A. Failure Attribution Rule

A rollout permits policy-behavior attribution only if its semantic, temporal, lifecycle, deployment, and evidence contracts pass. A policy-failure attribution additionally requires a retained failure layer identifying the policy. Otherwise the row is an integration, precondition, or evidence failure. Thus a missing actor proxy is not a weak planner, a command-proxy route is not a poor route-conditioned policy, and an incomplete audit is not a collision result. This rule permits policy behavior to be examined after validation; it does not declare every degraded metric a policy failure.

II. CONTRACT-GATED INTEGRATION

Table I maps the five recurring boundary mismatches to enforcement mechanisms. The *semantic contract* preserves route geometry, ego state, camera availability, and actor fields at prediction time. A high-level route command may be diagnostic context, but it cannot substitute for a local polyline. The *temporal contract* validates horizons and timestamps, anchors interpolation at the current ego pose, deterministically resamples a finite $N \times 2$ trajectory, and recomputes headings on the runtime grid.

The *lifecycle contract* makes start, active, close, cleanup, duplicate close, and late events explicit. Recoverable warnings are distinguished from fatal service errors. The *deployment contract* records launch commands, scene and policy identifiers, timeout, endpoint, repository state, image identity, and gated inputs. The *evidence contract* requires unique run identifiers, immutable manifests, terminal status, route and sensor audits, failed-run retention, and hashes linking generated paper assets to aggregate rows.

Figure 1 shows the boundary. WOD2Sim reconstructs policy-facing state, preserves route geometry, resamples

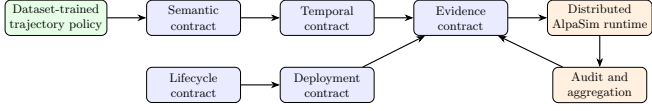


Fig. 1. Contract gates localize semantic, temporal, lifecycle, deployment, and evidence failures around the external-driver boundary.

outputs, and isolates optional model discovery. On the runtime side it records deployment and evidence state before a row can be attributed to policy behavior. Constant-velocity and route-following models form the public core. Learned-checkpoint and direct actor-aware entry points remain gated when their checkpoints or scene-matched actor proxies are unavailable.

III. IMPLEMENTATION AND EVIDENCE PIPELINE

A. Policy-Facing Adapter

The external driver maintains isolated state for each simulator session. Image, egomotion, route, and close messages update that state; prediction reads one consistent snapshot rather than reconstructing inputs from process-global variables. The route converter carries simulator waypoints into the policy input and records their source. When the explicit command-only diagnostic is selected, it labels the fallback as `command_proxy`; the audit can then reject the row even if the service and simulator finish normally. Static hazards remain geometry-bearing objects, while dynamic hazards retain position, motion, heading, and shape fields when the external interface supplies them. Visibility-risk signals remain diagnostics and do not create synthetic obstacles.

The temporal adapter maps a source trajectory to runtime times $t_k = k/f_{\text{run}}$ over the requested horizon. The current ego origin is prepended before interpolation, source timestamps must be finite and strictly increasing, and the output must be a finite $N \times 2$ array. Matching grids preserve samples; mismatched grids use deterministic interpolation, after which headings are recomputed from successive runtime points. Invalid horizons, frequencies, grids, or trajectories fail the temporal contract rather than reaching vehicle control. These behaviors are covered by dependency-light tests, but the scene-level temporal comparison remains blocked and is not reported as a result.

B. Artifact Lineage

Figure 2 shows the evidence path. Before launch, the matrix runner expands a configuration row and materializes its command, environment, scene identity, policy identity, adapter mode, prerequisite state, and expected route source. A readiness check distinguishes an executable row from a missing checkpoint, actor proxy, asset, or runtime dependency. Terminal state is then recorded as completed, failed, blocked, or planned; non-completed rows are retained rather than removed from the denominator.

For completed simulator rows, the audit reconstructs route-source and sensor-freshness status from retained telemetry and loads behavior metrics without using them to decide route validity. Aggregation rejects duplicate completed run

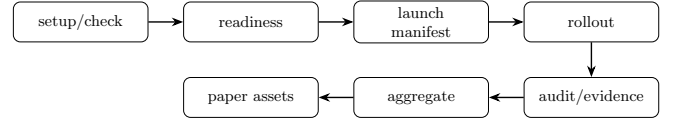


Fig. 2. Manifests and audits precede aggregation, so blocked or invalid rows remain visible and cannot silently become paper results.

identifiers, preserves failure codes, forms paired rows by policy, scene, and execution identifier, and writes CSV, JSON, LaTeX tables, and vector figures. Generated paper assets carry the aggregate data hash, and the submission validator compares their values with the JSON source. This lineage prevents manual table edits from changing a paper result independently of the row evidence. Restricted sensor frames and licensed scenes are not redistributed; the public package retains manifests, aggregate metrics, audit fields, and a stable frame schema. Consequently, artifact lineage is public, but complete replay still depends on legitimately acquired external scenes and runtime software.

IV. EVALUATION METHOD

The contract-validation matrix (CVM) contains 148 configured rows: 45 core, 30 semantic-ablation, 18 temporal-ablation, 40 lifecycle, and 15 fault-injection rows. It completes 115 rows, including 60 simulator rollouts; 33 optional-extension rows remain blocked. Lifecycle and fault rows use public service harnesses and are conformance diagnostics, not simulator-backed effectiveness results.

The public core executes two dependency-light policies over 15 locally cached front-camera scenes. The semantic matrix pairs full-route and command-only variants of route following on the same scenes. The launcher does not expose a simulator seed override, so each policy/scene arm has one execution; the configured seed is provenance metadata rather than proof of deterministic replay. No authoritative metadata verifies straight, intersection, lane-change, dense-traffic, occlusion, or merge labels: 15 scenes remain unclassified and 0/6 required categories are verified.

Completed rows are audited for route source and sensor freshness. For a precise comparison, define a status-only baseline

$$B_{\text{status}}(r) = \mathbf{1}[\text{completed}(r) \wedge \text{metrics}(r)]. \quad (1)$$

It models the minimal reporting path that scores any completed metric-bearing row; it is not a separate wrapper implementation or an external competitor. WOD2Sim additionally requires the five contracts. A semantic pair is comparison-eligible only when both arms contain metrics, the full-route arm is route-valid and audit-valid, and the command-only arm is route-invalid and audit-invalid. This criterion excludes one pair whose nominal full arm also fell back to a command proxy. Metric deltas are therefore computed on 14 pairs, but are not used for an inferential policy-effect claim.

V. RESULTS

Table II reports integration status rather than behavior scores. All 30/30 public-core rows complete with no service

TABLE II
DEPENDENCY-LIGHT PUBLIC-CORE INTEGRATION STATUS.

Public core policy	Rows	Done/att.	Audit	Route	Sensor	Crash	Blocked
constant_velocity	15	15/15	14/15	14/15	15/15	0	0
route_following	15	15/15	14/15	14/15	15/15	0	0

TABLE III
CLOSED-LOOP ATTRIBUTION CHECKS FROM RETAINED ROLLOUT AND
AUDIT ARTIFACTS.

Closed-loop check	Obs.	Denom.	Meaning
Full-contract audit-valid	42	45	audit passed
Comparison-eligible pairs	14	15	valid/invalid arms
Status-only baseline accepted	15	15	completion + metrics
Invalid route rows rejected	15	15	route gate
Verified scenario categories	0	6	coverage withheld

crash or blocker; 28 are audit-valid. The two invalid core rows and one invalid semantic full-route row come from the same scene, where the audit finds late command-proxy fallback. They remain integration/evidence failures despite successful process completion.

The central invalidation check is shown in Table III. All 15/15 command-only rows satisfy B_{status} , whereas WOD2Sim rejects 15/15 as route-invalid. This demonstrates a classification difference for retained closed-loop records: completion and metrics alone would admit them, while the semantic contract prevents policy attribution. It does not demonstrate improved driving or faster fault diagnosis.

Only 14/15 route-loss pairs satisfy the comparison criterion. Their full-minus-command score deltas have mixed signs and are descriptive only. With one uncontrolled execution per arm, the data do not support a systematic route-loss effect or any policy-quality comparison.

Across all retained rows, 42 are policy-behavior-attributable diagnostics, 0 are policy-failure-attributable, and 33 are integration/precondition blockers. The remaining 106 include blocked, contract-invalid, synthetic, or benchmark-incomplete evidence. These counts operationalize the distinction: degraded metrics cannot assign policy failure unless the contracts pass and the retained failure layer is policy.

A. Interpretation

The experiment validates an attribution mechanism, not a driving intervention. The positive path shows that 42 completed full-contract rows meet the recorded route and sensor checks. It does not establish a false-positive rate: audit-valid rows define the admitted set, and there is no independent oracle labeling which completed integrations should have passed. For that reason, WOD2Sim reports the directly observed 42/45 audit outcome and makes no “zero false blocks” claim.

The negative path asks a narrower counterfactual. If reporting used only terminal status and metric availability, every command-only row would be admitted. Adding route-source validity changes the attribution of all 15 such rows without changing their recorded simulator outcome. This is useful because the invalid records remain available for

TABLE IV
DESCRIPTIVE FULL-ROUTE MINUS COMMAND-ONLY MEAN DELTAS OVER
THE 14 ELIGIBLE PAIRS.

Metric	Mean delta
Progress	-0.052
Relative progress	-0.023
Collision-any	0.071
Off-road	0.000
Plan deviation	0.016

debugging while being excluded from policy evaluation. The gate neither repairs the route nor proves that a valid route would improve the score; Table IV in fact shows small and mixed average differences across metrics.

Operationally, the same pattern gives each unavailable extension a typed outcome rather than an omitted experiment. A missing actor proxy or checkpoint is recorded as a deployment precondition, a command proxy as a semantic violation, and missing audit state as an evidence violation. This partition makes a future learned-policy study additive: new rows can be admitted only after their prerequisites and audits pass, without rewriting the interpretation of the retained invalid or blocked rows.

VI. RELATED WORK AND LIMITATIONS

The contracts instantiate interface reasoning for cyber-physical systems [3], [4]. Closed-loop platforms and benchmarks such as CARLA, nuPlan, Waymax, NAVSIM, and AlpaSim [5], [6], [7], [8], [2] emphasize simulation or planning evaluation. Scenario and falsification tools such as Scenic, VerifAI, DriveFuzz, and PlanFuzz [9], [10], [11], [12] generate or mutate conditions. WOD2Sim is complementary: it asks whether the policy/simulator boundary preserved the assumptions needed to interpret a rollout.

This distinction is upstream of benchmark metric design. nuPlan and NAVSIM formalize planning tasks and evaluation measures, while Waymax and CARLA provide controllable simulation interfaces. AlpaSim provides the external-driver service boundary used here. Those systems can expose malformed or incomplete inputs, but their behavior metrics do not by themselves identify which integration assumption failed. Contract-based design offers a vocabulary for assumptions and guarantees; WOD2Sim applies that vocabulary as runtime admission and artifact-lineage checks. Scenic and falsification tools vary the scenario presented to a system, whereas the present semantic ablation deliberately varies the information preserved by the integration boundary. The two directions are orthogonal and could be combined once verified scenario metadata and a claim-ready policy matrix exist.

The limitations are substantial. The repository is not an official WOD interface and does not reconstruct complete WOD worlds or redistribute restricted scenes. It has no public learned checkpoint, no scene-matched actor proxy, no verified scenario-category coverage, no completed temporal ablation, and no simulator-backed lifecycle or fault-injection stress comparison. The status-only baseline is deliberately minimal,

not a competitive integration method. Restricted frame data are not public, and the public frame artifact is schema-only. Finally, one execution per arm and lack of runtime seed control preclude variance or significance estimates. These constraints block learned-policy, benchmark, coverage, reproducibility, and performance claims; the supported result is limited to contract enforcement and evidence attribution.

A. Threats to Validity

Internal validity. The command-only arm intentionally violates the declared semantic contract, so its rejection validates enforcement of a known condition rather than discovery of an unknown defect. The status-only baseline is evaluated on the same retained rows and ignores route and sensor fields by definition. It therefore isolates the attribution decision, but cannot support claims about runtime overhead, time-to-diagnosis, or superiority to another integration framework. In addition, the three invalid nominal full-contract rows share one scene and one observed fallback pattern; they are not three independent failure mechanisms.

Construct validity. Audit validity in the completed closed-loop rows directly measures route source and sensor freshness. Temporal correctness is exercised in unit tests, and lifecycle and fault localization are exercised in synthetic service harnesses. Those checks do not establish that all five contracts were stressed under the same simulator conditions. Counts of contract-valid rows therefore measure admission to the stated evidence boundary, not driving safety, integration completeness, or correctness of the simulator itself. Likewise, “policy-behavior-attributable” means no recorded boundary violation was found; it is not a claim-valid benchmark row.

External and conclusion validity. The scenes were selected by local asset availability, remain category-unclassified, and cannot represent the WOD distribution or rare-event coverage. Results from constant-velocity and route-following policies cannot be generalized to learned trajectory policies. Restricted assets also prevent a reviewer without the same licensed inputs from replaying the closed-loop rows, although the public aggregates and gate logic can be inspected. Finally, the paired sample has 14 eligible scenes, one execution per arm, and no effective simulator seed control. Mixed descriptive deltas cannot be treated as independent repeated trials or as evidence of a causal performance effect.

VII. CONCLUSION

WOD2Sim formulates a closed-loop policy boundary as semantic, temporal, lifecycle, deployment, and evidence contracts. The current release shows that its dependency-light path executes and that route-invalid, metric-bearing rows are withheld from policy attribution. It does not show policy

superiority, learned-policy performance, or scenario coverage. The defensible result is narrower: explicit contract gates make integration evidence auditable and prevent boundary or precondition failures from being reported as policy failures.

ACKNOWLEDGMENT

OpenAI Codex (GPT-5) assisted with language editing, consistency checks, and code audit. It did not generate experimental data; the author reviewed the resulting manuscript and accepts responsibility for its content.

REFERENCES

- [1] P. Sun, H. Kretschmar, X. Dotiwalla, A. Chouard, V. Patnaik, P. Tsui, J. Guo, Y. Zhou, Y. Chai, B. Caine, *et al.*, “Scalability in perception for autonomous driving: Waymo open dataset,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 2446–2454, 2020.
- [2] NVIDIA Corporation, “AlpaSim: A modular, lightweight, and data-driven research simulator for autonomous driving,” Software, 2025.
- [3] A. Sangiovanni-Vincentelli, W. Damm, and R. Passerone, “Taming dr. frankenstein: Contract-based design for cyber-physical systems,” *European Journal of Control*, vol. 18, no. 3, pp. 217–238, 2012.
- [4] L. de Alfaro and T. A. Henzinger, “Interface automata,” in *Proceedings of the 8th European Software Engineering Conference*, pp. 109–120, ACM, 2001.
- [5] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, “CARLA: An open urban driving simulator,” in *Proceedings of the 1st Annual Conference on Robot Learning*, vol. 78 of *Proceedings of Machine Learning Research*, pp. 1–16, PMLR, 2017.
- [6] H. Caesar, J. Kabzan, K. S. Tan, W. K. Fong, E. Wolff, A. Lang, L. Fletcher, O. Beijbom, and S. Omari, “nuPlan: A closed-loop ml-based planning benchmark for autonomous vehicles,” in *CVPR Workshop on Autonomous Driving*, 2021.
- [7] C. Gulino, J. Fu, W. Luo, G. Tucker, E. Bronstein, Y. Lu, J. Harb, X. Pan, Y. Wang, X. Chen, J. D. Co-Reyes, R. Agarwal, R. Roelofs, Y. Lu, N. Montali, P. Mouglin, Z. Yang, B. White, A. Faust, R. McAllister, D. Anguelov, and B. Sapp, “Waymax: An accelerated, data-driven simulator for large-scale autonomous driving research,” in *Advances in Neural Information Processing Systems*, 2023.
- [8] D. Dauner, M. Hallgarten, T. Li, X. Weng, Z. Huang, Z. Yang, H. Li, I. Gilitschenski, B. Ivanovic, M. Pavone, A. Geiger, and K. Chitta, “NAVSIM: Data-driven non-reactive autonomous vehicle simulation and benchmarking,” in *Advances in Neural Information Processing Systems*, 2024.
- [9] D. J. Fremont, T. Dreossi, S. Ghosh, X. Yue, A. L. Sangiovanni-Vincentelli, and S. A. Seshia, “Scenic: A language for scenario specification and scene generation,” in *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pp. 63–78, ACM, 2019.
- [10] T. Dreossi, D. J. Fremont, S. Ghosh, E. Kim, H. Ravanbakhsh, M. Vazquez-Chanlatte, and S. A. Seshia, “VerifAI: A toolkit for the formal design and analysis of artificial intelligence-based systems,” in *Computer Aided Verification*, pp. 432–442, Springer, 2019.
- [11] S. Kim, M. Liu, J. Rhee, Y. Jeon, Y. Kwon, and C. H. Kim, “DriveFuzz: Discovering autonomous driving bugs through driving quality-guided fuzzing,” in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, ACM, 2022.
- [12] Z. Wan, J. Shen, J. Chuang, X. Xia, J. Garcia, J. Ma, and Q. A. Chen, “Too afraid to drive: Systematic discovery of semantic DoS vulnerability in autonomous driving planning under physical-world attacks,” in *Proceedings of the Network and Distributed System Security Symposium*, Internet Society, 2022.